

Tersine Mühendislik ve Mühendislik Refleksi: Sistem Modellemesinden Örgütsel Öğrenmeye

Ömer Faruk Yavuz

Temmuz 2026

Özet

Bu çalışma, birbirinden bağımsız görünen iki alanı ortak bir epistemolojik zemin üzerinde birleştirmeyi amaçlamaktadır. Birinci alan, bir sistemin gözlemlenebilir davranışlarından hareketle iç modelinin yeniden kurulması anlamına gelen tersine mühendislik disiplindir. İkinci alan, mühendislik ekiplerinde belirsizliğin yönetimi, geri bildirim kültürü ve yanlış yolda olunduğunun erken kabulüne ilişkin örgütsel reflekslerdir. Çalışmanın temel savı, her iki alanın da aynı bilişsel ve yöntemsel disipline dayandığıdır: gözlemin yorumdan ayrılması, hipotezin doğrulanmaya değil yanlışlanmaya çalışılması ve mevcut modele duygusal bağlılık geliştirilmemesi. Birinci alanda bu disiplin dış sisteme, ikinci alanda ise çalışanın kendi çalışmasına ve ekibin kendi süreçlerine yöneltilmektedir.

Giriş

Problem ve Kapsam

Mühendislik pratiğinde karşılaşılan sistemlerin önemli bir bölümü, eksiksiz dokümantasyonla birlikte gelmez. Eski sistemler, üçüncü taraf bileşenler, tarihsel olarak biriken kod tabanları ve dokümantasyonu üretenlerden kopmuş kurumsal süreçler, mühendisi düzenli olarak aynı konuma yerleştirir: elde yalnızca davranış vardır, niyet ise yoktur.

Naur'un (1985) ortaya koyduğu üzere programlama, esasen metin üretimi değil kuram inşasıdır; kaynak kod, geliştiricinin zihnindeki kuramın kısmi ve kayıplı bir izdüşümüdür. Bu tez kabul edildiğinde tersine mühendisliğin niteliği de değişir: yapılan iş, kaybolmuş bir metnin geri getirilmesi değil, kaybolmuş bir kuramın yeniden inşasıdır. Polanyi'nin (1966) örtük bilgi kavramı aynı sonucu destekler; bir sistemin nasıl çalıştığına dair bilginin önemli bölümü hiçbir zaman yazıya dökülmemiştir.

Bu çalışmanın kapsamı, meşru ve yetkilendirilmiş tersine mühendislik faaliyetleriyle sınırlıdır. Erişim denetimlerinin aşılması, lisans veya teknolojik koruma önlemlerinin devre dışı bırakılması ve üçüncü kişilere ait sistemlere izinsiz erişim, kapsam dışında tutulmuş; ilgili hukuki çerçeve ayrı bir bölümde ele alınmıştır.

Doküman Yapısı

Birinci bölüm, tersine mühendisliğin epistemolojik temellerini ve kuramsal sınırlarını incelemektedir. İkinci bölüm temel prensipleri, üçüncü bölüm yöntemleri, dördüncü bölüm süreç modelini, beşinci bölüm iyi uygulamaları, altıncı bölüm hukuki ve etik çerçeveyi ele almaktadır. Yedinci bölüm, aynı epistemik tutumun örgütsel düzlemdeki karşılığını örgütsel öğrenme yazını üzerinden tartışmakta; sekizinci bölüm iki düzlem arasındaki yapısal karşılığı sentezlemektedir.

Bölüm I: Epistemolojik Temeller

Tanım ve Yön

Chikofsky ve Cross'un (1990) yerleşik hâle gelen tanımına göre tersine mühendislik, bir sistemin bileşenlerini ve aralarındaki ilişkileri belirlemek ve sistemi başka bir biçimde ya da daha yüksek bir soyutlama düzeyinde temsil etmek amacıyla incelenmesi sürecidir. Aynı çalışmada bu faaliyet, *tasarım kurtarma* (design recovery), *yeniden dokümantasyon* (redocumentation), *yeniden yapılandırma* (restructuring) ve *yeniden mühendislik* (reengineering) kavramlarından ayrıştırılmıştır. Kritik nokta, tersine mühendisliğin kendi başına bir değişiklik faaliyeti olmasıdır; salt bir inceleme ve temsil üretme faaliyetidir.

İleri yönlü mühendislikte üretim zinciri şu biçimdedir:

$$Gereksinim \rightarrow Tasarım \rightarrow Uygulama \rightarrow Ürün$$

Tersine mühendislikte çıkarım zinciri bu yönün tersine işler:

$$Ürün \rightarrow Davranış \rightarrow Yapı \rightarrow Tasarımniyeti$$

Bu ters yönlü hareketin belirleyici özelliği bilgi kaybıdır. Derleme, optimizasyon ve dağıtım süreçleri; değişken adlarını, yorum satırlarını, tip bilgisini ve en önemlisi tasarım gerekçesini geri döndürülemez biçimde ortadan kaldırır. Dolayısıyla amaç, orijinal yapıya birebir ulaşmak değil, belirlenen soru kümesini yanıtlamaya yeterli bir model üretmektir.

Abdüktif Çıkarım Olarak Tersine Mühendislik

Tersine mühendislikte kullanılan çıkarım biçimi ne tümdengelim ne de basit tümevarımdır. Peirce'ün *abdüksiyon* olarak adlandırdığı, şaşırtıcı bir gözlemden onu açıklayabilecek en makul hipoteze geçiş biçimidir (Josephson ve Josephson, 1994). Abdüktif çıkarımın yapısı şöyledir: *O* gözlemlenmiştir; eğer *H* doğru olsaydı *O* olağan olurdu; dolayısıyla *H* doğru olabilir.

Bu çıkarım biçiminin mantıksal olarak geçersiz olduğu (sonucun ardbileşenin onaylanması hatasını taşıdığı) açıktır. Bu nedenle abdüksiyon tek başına kullanıldığında güvenilir değildir; ancak sistematik yanlışlama ve çoklu hipotez yönetimiyle birleştirildiğinde güçlü bir keşif aracına dönüşür. Bu gözlem, ilerideki bölümlerde ele alınan yanlışlama ve gözlem matrisi uygulamalarının neden bir tercih değil zorunluluk olduğunu açıklamaktadır.

Modelin Yeterlilik Ölçütü

Box'ın (1976) sıkça anılan formülasyonu bu alanda doğrudan geçerlidir: bütün modeller yanlıştır, bir kısmı yararlıdır. Tersine mühendislikte üretilen model için doğruluk ölçütü, orijinal tasarımla özdeşlik değil, öngörü gücüdür. Bir modelin yeterliliği şu ölçüte bağlanabilir:

Model M , sistem S için, ilgilenilen girdi uzayı I üzerinde, $\forall i \in I$ için $M(i) \approx S(i)$ sağlıyorsa yeterlidir.

Buradaki I kümesinin sınırlarının açıkça belirtilmesi zorunludur. Kapsam dışı girdilerde modelin öngörü üretmeyeceği, dokümantasyonun açık bir unsuru olmalıdır.

Kanıtın Kuramı Belirlemedeki Yetersizliği

Duhem-Quine tezinin ortaya koyduğu üzere, sonlu bir gözlem kümesiyle uyumlu sonsuz sayıda kuram kurulabilir. Tersine mühendislikte bunun pratik karşılığı şudur: gözlemlenen davranışı açıklayan birden fazla iç yapı her zaman mevcuttur. Aynı çıktıyı üreten farklı algoritmalar, aynı hata mesajını üreten farklı denetim mekanizmaları, aynı byte dizisini üreten farklı serileştirme şemaları bulunabilir.

Bu durumun yöntemsel sonucu, Chamberlin'in (1890) *çoklu çalışma hipotezleri* yönteminin benimsenmesidir. Chamberlin, tek bir açıklamaya erken bağlanmanın araştırmacıyı o açıklamanın savunucusuna dönüştürdüğünü ve gözlemi çarpıttığını göstermiştir. Önerdiği çözüm, aynı anda birden fazla hipotezin eşit ciddiyetle taşınmasıdır. Platt (1964) aynı yöntemi *güçlü çıkarım* adı altında sistemleştirmiş; her adımda hipotezler arasında ayırım yaratacak deneyin (*experimentum crucis*) tasarlanmasını önermiştir.

Kuramsal ve Pratik Sınırlar

Tersine mühendisliğin sınırları yalnızca pratik değil, kuramsaldır.

Rice teoremi, bir programın önemsiz olmayan herhangi bir anlamsal özelliğinin genel durumda karar verilebilir olmadığını ortaya koyar. Bunun doğrudan sonucu, tam ve otomatik statik analizin genel durumda imkânsız olmasıdır. Bu nedenle pratik statik analiz araçları, Cousot ve Cousot'nun (1977) soyut yorumlama çerçevesinde formelleştirdiği biçimde, kesinlikten ödün vererek karar verilebilirlik kazanır; sonuç, aşırı yaklaşık (over-approximate) ya da eksik yaklaşık (under-approximate) analizlerdir.

İkinci sınır kasıtlıdır. Collberg, Thomborson ve Low'un (1997) taksonomisinde sınıflandırıldığı üzere, karartma (obfuscation) teknikleri; düzen dönüşümleri, veri dönüşümleri ve denetim akışı dönüşümleri yoluyla analiz maliyetini sistematik olarak artırır. Paketleme, sanallaştırma tabanlı koruma, hata ayıklayıcı karşıtı teknikler ve kendini değiştiren kod bu kategoriye girer.

Üçüncü sınır, dinamik analizin doğasından kaynaklanır: yalnızca yürütülen yollar gözlemlenebilir. Kod kapsamı eksik kaldığı sürece, gözlemlenmemiş davranışların varlığı dışlanamaz.

Bölüm II: Temel Prensipler

Gözlem ile Yorumun Ayrılması

Analizin en erken aşamasında yapılan en yaygın hata, ham bulgunun yorumla birleştirilerek kaydedilmesidir. Bir uç noktanın belirli bir istekte hata üretmesi bir gözlemdir; arka planda belirli bir veritabanı bulunduğu ise bir yorumdur. İkisi aynı kayıta birleştirildiğinde, yorum zamanla gözlemin yerini alır ve son-raki analizler yanlış bir temel üzerine kurulur.

Ayrım şu biçimde formülleştirilebilir:

$$\text{Gözlem} \neq \text{Yorum}$$

Heuer'in (1999) istihbarat analizi bağlamında geliştirdiği *Rakip Hipotezler Analizi* (Analysis of Competing Hypotheses) yöntemi bu ayrımı operasyonel hâle getirir. Yöntem, kanıtların hipotezleri *doğrulama* gücüne göre değil, hipotezleri *eleme* gücüne göre değerlendirilmesini öngörür. Aynı kanıt tüm hipotezlerle uyumluysa tanısal değeri sıfırdır.

Kara Kutu Yaklaşımı

Kara kutu kavramının kuramsal kaynağı sibernetiktir. Ashby (1956), gözlemcinin yalnızca girdi ve çıktıya erişebildiği bir sistemin nasıl incelenebileceğini sistem-atik biçimde ele almış ve gözlemcinin ancak sistemin ayırt edebildiği kadar durum ayırt edebileceğini göstermiştir. Aynı çerçevede formüle edilen *gerekli çeşitlilik yasası*, bir denetleyicinin ancak denetlediği sistemin çeşitliliği kadar çeşitliliğe sahipse etkili olabileceğini ifade eder; tersine mühendislikteki karşılığı, deney tasarımının çeşitliliğinin modelin çözünürlüğünü sınırlamasıdır.

Kara kutu aşamasında yanıtlanması gereken sorular şunlardır:

- Sistem hangi girdileri kabul etmektedir?
- Hangi çıktıları üretmektedir?
- Hangi koşullarda hata üretmektedir?
- Durum (state) tutmakta mıdır?
- Zamanlama davranışı nasıldır?
- Hangi dış sistemlerle haberleşmektedir?

Kimlik doğrulama davranışına ilişkin örnek bir gözlem tablosu:

Girdi	Çıktı	Değerlendirme
Geçersiz parola	Genel hata mesajı	Ayırt edici bilgi verilmiyor
Var olmayan kullanıcı	Aynı mesaj	Kullanıcı numaralandırması engellenmiş olabilir
Yoğun deneme	Artan gecikme	Hız sınırlama veya kilitleme olabilir

Burada üçüncü sütunun “çıkarım” değil “değerlendirme” olarak adlandırılması ve olasılık kipiyle yazılması, bir önceki bölümdeki ayrımın doğrudan uygulamasıdır.

Ayrıştırma

Simon’un (1962) karmaşıklık mimarisi üzerine çalışması, karmaşık sistemlerin büyük çoğunluğunun *yaklaşık ayrıştırılabilir* (nearly decomposable) olduğunu; yani alt sistem içi etkileşimlerin alt sistemler arası etkileşimlerden belirgin biçimde daha yoğun olduğunu ortaya koymuştur. Bu özellik, ayrıştırmanın yalnızca bir kolaylık değil, sistemin gerçek yapısına karşılık gelen bir strateji olmasını sağlar.

Genel ayrıştırma şeması:

$$Sistem = Girdi + İşleme + Durum + Çıktı + Bağımlılıklar$$

Parnas’ın (1972) modülerlik ölçütü, ayrıştırmanın hangi ekseninde yapılması gerektiğine dair ek bir yön verir: modül sınırları işlem adımlarına göre değil, gizlenen tasarım kararlarına göre çizilmelidir. Tersine mühendislikte bunun karşılığı, “değişme olasılığı yüksek olan bilgi nerede kapsüllenmiş?” sorusudur; bu soru genellikle mimarinin gerçek eklem yerlerini ortaya çıkarır.

Bir masaüstü uygulaması için tipik ayrıştırma eksenleri: kullanıcı arayüzü, dosya sistemi kullanımı, ağ trafiği, yerel veri saklama, iş mantığı, harici servis bağımlılıkları, güncelleme mekanizması ve hata yönetimi. Gömülü bir cihaz için: güç devresi, sensörler, mikrodenetleyici, haberleşme hatları, bellek, aktüatörler ve mekanik bileşenler.

Kontrollü Tek Değişken İlkesi

Yöntemin özü, aynı deneyin yalnızca tek bir değişken değiştirilerek yinelenmesidir:

$$\Delta Girdi \rightarrow \Delta Çıktı$$

Bir dosya biçiminin çözümlenmesinde izlenen tipik yordam:

1. Asgari içerikli bir belge oluşturulur ve kaydedilir.
2. Tek bir alan değiştirilir.
3. Belge yeniden kaydedilir.
4. İki çıktı ikili düzeyde karşılaştırılır.
5. Değişen bölge, değiştirilen alanla ilişkilendirilerek hipotez kurulur.

Bu sezgisel yordamın biçimsel karşılığı Zeller ve Hildebrandt’ın (2002) geliştirdiği *delta hata ayıklama* (delta debugging) algoritmasıdır. Algoritma, bir hatayı tetikleyen girdiyi sistematik ikiye bölme yoluyla asgari tetikleyici alt küme indirger ve böylece elle yapılan tek değişken denemesini otomatikleştir. Zeller’in (2009) daha geniş çerçevesi, hata ayıklamayı açıkça bilimsel yöntemin bir uygulaması olarak konumlandırır: gözlem, hipotez, öngörü, deney, yineleme.

Yanlışlama İlkesi

Popper'ın (1959) bilim felsefesindeki temel ölçütü, bir önermenin bilimsel niteliğinin doğrulanabilirliğinde değil yanlışlanabilirliğinde bulunmasıdır. Tersine mühendislikte bu ölçüt doğrudan uygulanabilir: kurulan her hipotez, onu çürütebilecek somut bir deneyle birlikte kaydedilmelidir.

“Dosyanın ilk dört baytı kayıt sayısını tutmaktadır” biçimindeki bir hipotez için doğrulayıcı deneyler yetersizdir. Çürütme potansiyeli taşıyan deneyler şunlardır: sıfır kayıt, tek kayıt, 255 kayıt, 256 kayıt (bayt taşması sınırı), 65 535 ve 65 536 kayıt (16 bit sınırı), kayıt silme, belgenin yeniden açılıp yeniden kaydedilmesi, farklı sürümle kaydedilmesi.

Sorulacak soru “bu hipotez doğru mu?” değil, “bu hipotez hangi koşulda çöker?” biçiminde formüle edilmelidir.

Lakatos'un (1970) araştırma programları kuramı bu tabloya önemli bir nüans ekler. Hiçbir hipotez tek bir olumsuz gözlemle terk edilmez; çevresindeki yardımcı varsayımlardan oluşan koruyucu kuşak, hipotezi ayakta tutmak için sürekli tadil edilebilir. Ayırt edici ölçüt, bu tadillerin yeni öngörüler üretip üretmediğidir. Yalnızca mevcut anomaliyi kapatmaya yarayan, yeni hiçbir öngörü üretmeyen ardışık tadiller, *gerileyen* bir program işaretidir. Bu ölçüt, yedinci bölümde ele alınan “çıkamaz sokak” tanısının kuramsal karşılığıdır.

Soyutlama Katmanlarının Ayrılması

Dijkstra'nın (1972) vurguladığı üzere, soyutlamanın işlevi belirsizlik yaratmak değil, doğru olunabilecek yeni bir anlam düzeyi kurmaktır. İnceleme faaliyeti de bu düzeylere saygı göstermek zorundadır.

Katman	İnceleme nesnesi
Kullanıcı düzeyi	Ekranlar, akışlar, iş kuralları
Uygulama düzeyi	Modüller, sınıflar, fonksiyonlar
İşletim sistemi düzeyi	Dosyalar, süreçler, izinler, sistem çağrıları
Ağ düzeyi	Paketler, protokoller, uç noktalar
Makine düzeyi	Komutlar, yazmaçlar, bellek düzeni
Donanım düzeyi	Sinyaller, tümleşik devreler, veri yolları

İlke şudur: soru, yanıtlanabileceği en yüksek soyutlama düzeyinde ele alınmalı; yalnızca o düzeyde çözülemediğinde bir alt katmana inilmelidir. Katman atlamamanın maliyeti yalnızca zaman değil, dikkatin dağılmasıyla oluşan kavram kirliliğidir.

Ekonomi İlkesi

Bir ikili dosyada dört baytın değişmiş olması, ilk açıklama olarak şifreleme varsayımını gerektirmez. Sıralı biçimde önce şu basit açıklamalar elenmelidir: uzunluk alanı, sayaç, zaman damgası, sağlama toplamı, kimlik alanı, görelî konum (offset), sürüm numarası, sıkıştırma farkı, hizalama dolgusu, rastgele tohum.

Bu, parsimoni ilkesinin (Occam usturası) yöntemsel uygulamasıdır: karmaşık açıklama, basit açıklamalar deneysel olarak elendikten sonra gündeme alınmalıdır. İlkenin gerekçesi metafizik değil ekonomiktir; basit hipotezler daha az deneyle çürütülebilir olduğundan arama uzayını daha hızlı daraltır.

Bölüm III: Yöntemler

Statik Analiz

Statik analiz, sistem yürütülmeden gerçekleştirilen incelemedir. Başlıca teknikleri: dosya biçimi ve başlık incelemesi, karakter dizisi çıkarımı, bağımlılık ve içe aktarım tablosu analizi, sembol tablosu incelemesi, sökme (disassembly), geri derleme (decompilation), denetim akış grafiği çıkarımı ve veri yapısı kurtarma.

Güçlü yanı, sistemin bütününe eş zamanlı erişim sağlamasıdır; zayıf yanı, hangi yolların gerçekte yürütüldüğünün belirlenememesi ve Rice teoremi gereği genel durumda kesinlik sağlanamamasıdır (Eilam, 2005).

Dinamik Analiz

Dinamik analiz, sistem yürütülürken gerçekleştirilen gözlemdir. Başlıca teknikleri: kayıt (log) incelemesi, hata ayıklayıcı kullanımı, fonksiyon çağrısı izleme, sistem çağrısı izleme, dosya ve kayıt defteri erişimlerinin gözlemi, bellek anlık görüntüsü analizi, ağ trafiği çözümlemesi ve enstrümantasyon.

Güçlü yanı, fiili davranışın doğrudan gözlemlenmesidir; zayıf yanı, kapsamın yürütülen senaryolarla sınırlı kalmasıdır (Sikorski ve Honig, 2012).

En verimli sonuç, iki yaklaşımın birleşiminden elde edilir:

$$\textit{StatikAnaliz} + \textit{DinamikAnaliz}$$

Bu birleşimin biçimsel bir uygulaması, sembolik yürütme ile somut yürütmeyi birlikte kullanan *concolic* test yöntemidir; girdi uzayının sistematik olarak keşfedilmesini ve kod kapsamının artırılmasını sağlar.

Diferansiyel Analiz

İki durum arasındaki farkın izole edilmesine dayanır. Tipik karşılaştırma eksenleri: uygulamanın iki ardışık sürümü, boş ile dolu belge, başarılı ile başarısız istek, çalışan ile arızalı cihaz, bir özelliğin etkin ve devre dışı hâllerinde oluşan trafik, iki farklı yapılandırma altındaki davranış.

Yöntemin gücü, büyük hacimli veri içinden anlamlı değişimin ayıklanmasını sağlamasıdır. Yamalar üzerinden güvenlik açığı analizinde (patch diffing) ve dosya biçimi çözümlemesinde başat yöntemdir.

Davranışsal Modelleme ve Otomat Öğrenimi

Sistemin iç yapısından bağımsız olarak, gözlemlenen davranışın bir durum makinesi biçiminde temsil edilmesidir. Basit bir oturum modeli:

```
Baslangic
|
Kimlik dogrulanmamis
|  login basarili
Kimlik dogrulanmis
|  zaman asimi
Oturum sona ermis
```

Bu yaklaşımın biçimsel karşılığı, Angluin'in (1987) L^* algoritmasıyla kurduğu *aktif otomat öğrenimi* çerçevesidir. Algoritma, sisteme üyelik sorguları ve denklik sorguları göndererek gözlemlenen dile karşılık gelen asgari deterministik otomatı türetir. Vaandrager'ın (2017) derlemesinde gösterildiği üzere yöntem; ağ protokollerinin, akıllı kart uygulamalarının ve gömülü denetleyicilerin tersine mühendisliğinde başarıyla kullanılmıştır. Yaklaşımın sınırı, denklik sorgularının pratikte ancak yaklaşık olarak gerçekleştirilebilmesidir.

Veri Akışı Analizi

Bir bilginin kaynağından tüketimine kadar izlenmesidir:

$$Girdi \rightarrow Dönüşüm \rightarrow Saklama \rightarrow Kullanım \rightarrow Çıktı$$

Weiser'in (1981) tanımladığı *program dilimleme* (program slicing), belirli bir değişkenin belirli bir noktadaki değerini etkileyen tüm komutların çıkarılmasını sağlayarak bu izlemeyi biçimselleştirir. Çalışma zamanındaki karşılığı, verinin kaynaktan hedefe akışını etiketleyerek izleyen kirlilik analizidir (taint analysis).

Bir indirme işleminin ayrıştırılmış akışı, her düğümü bağımsız olarak gözlemlenebilir bir hipotez oluşturur: kullanıcı eylemi, arayüz olayı, istemci tarafı doğrulama, ağ isteği, sunucu tarafı yetkilendirme, nesne deposundan erişim, denetim kaydı yazımı, yanıtın istemciye ulaşması.

Protokol Analizi

İki sistem arasındaki iletişim biçiminin çözümlenmesidir. İnceleme unsurları: mesaj sınırları, başlık alanları, uzunluk göstergeleri, kimlik alanları, sıra numaraları, istek-yanıt eşleşmesi, hata kodları, durum geçişleri, karakter kodlaması, sıkıştırma ve bütünlük denetimleri.

Temel yöntem yine kontrollü değişiklikler: aynı mesaj yeniden gönderilir, tek bir alan değiştirilir ve karşı tarafın tepkisi kaydedilir. Alan sınırlarının otomatik çıkarımı üzerine geliştirilen yaklaşımlar arasında, yürütme izlerinden mesaj biçimi türeten Polyglot (Caballero ve diğerleri, 2007) ve yalnızca ağ izlerinden biçim çıkaran Discoverer (Cui, Kannan ve Wang, 2007) sayılabilir.

Protokol analizinde dikkate alınması gereken bir yapısal olgu, Postel'in sağlamlık ilkesidir (gönderimde katı, kabulde hoşgörülü olma): uygulamaların belgelenmemiş toleransları, gözlemlenen davranışın belgelenen biçimden sapmasına yol açar. Bu nedenle gözlemlenen davranış, belgelenmiş şartnamenin yerine değil yanına konularak kaydedilmelidir.

İkili ve Makine Düzeyi Analiz

Derlenmiş yazılımda yeniden kurulmaya çalışılan unsurlar: fonksiyon sınırları, çağrı sözleşmeleri, parametreler, dönüş değerleri, koşul ve döngü yapıları, bileşik veri yapıları, sanal fonksiyon tabloları ve nesne ilişkileri, sistem çağrıları, istisna ve hata yolları.

Örnek bir karşılaştırma ve dallanma dizisi:

```
cmp eax, 10
jle valid
```

yaklaşık olarak şu yüksek düzey yapıya karşılık gelir:

```
if (value <= 10) {
    // valid
}
```

Ancak geri derleyici çıktısının orijinal kaynak kod olmadığı sürekli akılda tutulmalıdır. Adlandırma, yorum satırları, tip anlamları ve tasarım gerekçesi geri döndürülemez biçimde kaybolmuştur; geri derleyicinin ürettiği yapı, derleyicinin optimizasyon kararlarının bir izdüşümüdür.

Donanım Düzeyi Analiz

Fiziksel sistemlerde uygulanan yöntemler ayrı bir küme oluşturur: hata ayıklama arayüzlerinin (JTAG, SWD, UART) tespiti, bellek yongalarının okunması, baskı devre kartının katman katman çıkarılması, tümleşik devrenin kapsül açımı ve optik incelenmesi, mantık analizörüyle veri yolu izleme.

Bu kümeye ayrıca yan kanal analizi dahildir. Kocher'in (1996) zamanlama saldırıları ve Kocher, Jaffe ve Jun'un (1999) diferansiyel güç analizi üzerine çalışmaları, sistemin işlevsel arayüzü dışındaki fiziksel gözlemlerin de bilgi sızdırdığını göstermiştir. Yöntemsel açıdan bu, kara kutu kavramının sınırlarının gözlemcinin ölçüm yeteneğiyle belirlendiğini örnekler.

Bölüm IV: Süreç Modeli

Amacın Daraltılması

“Sistemi tamamen çözmek” ölçülemez ve sonlanamaz bir hedeftir. İşlevsel hedef örnekleri: belirli bir dosya biçiminin okunabilmesi, eski bir sistemle uyumlu istemci geliştirilmesi, bir arızanın kaynak modülünün belirlenmesi, bir cihazın denetim sinyalinin çözülmesi, kaybolmuş dokümantasyonun yeniden üretilmesi, mevcut sistemin yerine geçecek bir uygulamanın şartnamesinin çıkarılması.

Hedefin niteliği, gerekli soyutlama derinliğini belirler. Simon'un (1956) *yet-inmecilik* (satisficing) kavramı burada doğrudan geçerlidir: aranan şey en iyi model değil, belirlenen ölçütü karşılayan ilk yeterli modeldir.

Referans Davranışın Belgelenmesi

Herhangi bir müdahaleden önce sistemin olağan çalışması kayıt altına alınır: girdiler, çıktılar, ekran görüntüleri, üretilen dosyalar, kayıt defterleri, ağ istekleri, zamanlama ölçümleri, hata mesajları, sürüm bilgileri, çalışma ortamı değişkenleri ve yapılandırma dosyaları.

Bu kayıt kümesi, sonraki tüm diferansiyel analizlerin referans çizgisini (baseline) oluşturur. Referans çizgisinin eksikliği, gözlemlenen her farkın müdahaleye mi yoksa ortama mı ait olduğunun ayrılmasına yol açar.

Gözlem Matrisinin Tutulması

No	Değiştirilen	Gözlenen	Hipotez	Güven
1	Kullanıcı adı	Hata mesajı değişmedi	Kullanıcı numaralandırması engelli	Yüksek
2	Dosya adı	8 bayt değişti	Ad, üstveride saklanıyor	Yüksek
3	Kayıt sayısı	12–15. baytlar değişti	32 bit sayaç	Orta
4	Oturum süresi	Yanıt gecikti	Sunucu tarafı zaman aşımı	Düşük

Matrisin işlevi çift yönlüdür. Birincisi bellek işlevidir; birkaç saat sonra hangi çıkarımın hangi kanıtı dayandığı hatırlanmaz. İkincisi denetim işlevidir; her satırın bir kanıt sütunu taşıması, kanıtsız çıkarımların görünür kalmasını sağlar.

Sistem Haritasının Çıkarılması

Önce kaba bir topoloji kurulur:

```
Kullanıcı arayuzu
|
Yerel servis
|
REST API
|
Veritabanı + Nesne deposu
```

Ardından her kutu aşamalı olarak açılır. Erken aşamada ayrıntıya inilmesi, sistemin ana damarlarının gözden kaçmasına yol açar. Haritanın ilk sürümünün işlevi doğruluk değil, yönelim sağlamaktır.

Güven Düzeylerinin Belirtilmesi

Sherman Kent'in (1964) istihbarat tahminlerinde kullanılan ifadelerin belirsizliği üzerine yaptığı çalışma, sözel olasılık ifadelerinin kişiler arasında tutarsız yorumlandığını göstermiş ve standartlaştırılmış bir terminoloji önermiştir. Aynı disiplin tersine mühendislik dokümantasyonunda da gereklidir:

- **Doğrulanmış:** Uygulama açılışta /`config` uç noktasına istek göndermektedir (ağ izinde on kez gözlemlenmiştir).
- **Yüksek olasılık:** Dönen belirteç JWT biçimindedir (yapı ve kodlama uyumludur, imza doğrulanmamıştır).
- **Orta olasılık:** Belirteç içindeki rol alanı arayüz yetkilerini belirlemektedir (iki rol için gözlemlenmiştir).
- **Düşük olasılık:** Aynı rol denetimi sunucu tarafında da zorunlu tutulmaktadır (doğrudan kanıt bulunmamaktadır).

Tetlock ve Gardner'ın (2015) öngörü doğruluğu üzerine bulguları, güven düzeyinin açıkça ifade edilmesinin yalnızca iletişim değil, tahmin kalitesi üzerinde de iyileştirici etki yarattığını göstermektedir.

Modelin Bağımsız Doğrulanması

Kurulan modelin sınanması, onu bağımsız olarak yeniden uygulamakla gerçekleşir: çözümlenen bir protokol için asgari bir istemci, çözümlenen bir dosya biçimi için bir ayrıştırıcı, çözümlenen bir algoritma için temiz bir uygulama yazılır.

Ölçüt şudur: model, oluşturulurken kullanılmayan girdilerde de doğru öngörü üretmekte midir? Bu, makine öğrenmesindeki genelleme ölçütüyle yapısal olarak aynıdır; gözlem kümesine aşırı uyum (overfitting), tersine mühendislikte de geçerli bir başarısızlık biçimidir.

Uygulamada bu doğrulama, orijinal sistemle yeni uygulamanın aynı girdi kümesi üzerinde karşılaştırıldığı diferansiyel test yoluyla yürütülür.

Bölüm V: İyi Uygulamalar

İzolasyon

Analiz hiçbir koşulda tek nüsha üzerinde yürütülmez. Kopya, anlık görüntü veya yalıtılmış sanal ortam kullanılır. Gerekçesi: analiz sırasında dosya bozulması, yapılandırma değişikliği, sistemin beklenmedik veri yazması, lisans aktivasyonunun tüketilmesi ve geri alınamaz durum değişikliği ihtimalleri mevcuttur. Bilinmeyen ikili dosyaların incelenmesinde ayrıca ağdan yalıtım zorunludur.

Tekrarlanabilirlik

İkinci kez üretilmeyen bir bulgu, bulgu sayılmaz. Kaydedilmesi gerekenler: işletim sistemi ve sürümü, hedef sistemin sürümü, tarih ve saat, kullanılan girdi dosyasının sağlama değeri, çalıştırılan komut, çıktının sağlama değeri, etkin yapılandırma ve ortam değişkenleri.

Bu gereklilik, Parnas ve Clements'in (1986) "rasyonel tasarım sürecinin sahtesini yapmak" önerisiyle aynı mantığa dayanır: süreç fiilen düzensiz ilerlese bile, çıktı belgesi başkasının izleyebileceği düzenli bir yapıda sunulmalıdır.

Otomasyon

Yinelenen karşılaştırmalar elle yürütülmez. Küçük araçlarla otomatikleştirilmesi gereken işler: ikili dosya farkı, onaltılık karşılaştırma, trafik özetleme, kayıt filtreleme, deney matrisi üretimi, çıktı sağlama hesabı, toplu girdi üretimi.

Tersine mühendislikte, sağlam kurulmuş küçük araçlar kapsamlı kuramsal modellerden çoğu kez daha yüksek getiri sağlar; çünkü deney döngüsünün süresini kısaltır ve döngü süresi, öğrenme hızını doğrudan belirler.

Adlandırmanın Kademeli İyileştirilmesi

Başlangıçta üretilen adlar bilgi taşımaz: `func_0045A120`, `field_08`, `unknown_packet_3`. Anlam çözüldükçe bunlar `parseUserRecord`, `recordCount`, `authenticationResponse` biçimini alır.

Adlandırmanın işlevi yalnızca okunabilirlik değildir; zihinsel modelin gelişiminin ölçülebilir göstergesidir. Bir bölgede hâlâ anlamsız adlar bulunuyorsa, o bölge henüz anlaşılmamıştır.

Bilinen ile Varsayılanın Ayrılması

Dokümantasyon şu bölümleri ayrı başlıklar altında tutmalıdır: gözlemler, kanıtlar, çıkarımlar, açık sorular, çelişkiler, önerilen deneyler. Ayrım yapılmadığında, bir kişinin varsayımı birkaç hafta içinde “sistemin bilinen gerçeği” hâline gelir ve bu dönüşüm belgede iz bırakmaz.

Bu, Bölüm II’de ele alınan gözlem–yorum ayrımının doküman düzeyindeki kurumsallaşmasıdır.

Temiz Oda Yöntemi

Uyumlu bir ürün geliştirilecekse uygulanan yöntem, iki ekibin bilgi akışının denetimli biçimde ayrılmasına dayanır:

- **Analiz ekibi:** Mevcut sistemin davranışını inceler, işlevsel şartname üretir, orijinal koda veya korunan ifadeye ilişkin hiçbir materyali diğer ekibe aktarmaz.
- **Uygulama ekibi:** Yalnızca şartnameyi görür, orijinal sisteme hiç maruz kalmamıştır ve uygulamayı sıfırdan yazar.

Amaç, korunan ifadeyi kopyalamadan davranışsal uyumluluk üretmektir. Yöntemin en bilinen uygulaması, IBM PC BIOS’unun uyumlu bir sürümünün geliştirilmesi sürecidir. Kullanım alanları: eski sistemlerle bütünleşme, dosya biçimi uyumluluğu, protokol uygulaması ve sistem değiştirme projeleridir.

Yöntemin değeri yalnızca teknik değil ispat hukukuna ilişkindir; bilgi akışının belgelenmiş ayrımı, kopyalama iddiasına karşı kanıt oluşturur.

İyi Bir Analistin Sorduğu Sorular

1. Sisteme giren bilgi hangi kaynaktan gelmektedir?
2. Sistemde hangi durumlar, nerede ve ne kadar süre tutulmaktadır?
3. Hangi davranış deterministiktir, hangisi rastlantısal veya zamana bağlıdır?
4. Hangi bileşen işlevsel olarak zorunludur, hangisi yalnızca optimizasyondur?
5. Hangi alan kimlik taşımaktadır, hangisi yalnızca görüntüleme amaçlıdır?
6. Hata nerede oluşmakta, nerede kullanıcıya çevrilmektedir?
7. İki işlem arasında gözlemlenmeyen bir durum aktarımı var mıdır?
8. Bu bulguyu hangi deney çürütebilir?
9. Gözlemlenen davranış istemciye mi sunucuya mı aittir?
10. Sonuç sürüme, ortama veya yapılandırmaya bağımlı mıdır?
11. Bu gözlem hangi hipotezleri elemektedir, hangilerini elemekte yetersizdir?
12. Modelin kapsam sınırı nerededir?

Özet Döngü

Tüm süreç şu yinelemeli döngüye indirgenebilir:

Gözlemle → Ayrıştır → Hipotez kur → Tekdeğiskenle deney yap → Karşılaştır → Model güncelle

Döngünün kapanış ölçütü şudur: anlaşıldığı düşünülen şey yeniden üretilmiyorsa, henüz anlaşılmamıştır.

Bölüm VI: Belirsizlik, Geri Bildirim ve Örgütsel Refleks

Belirsizliğin Sorumluluk Dağılımı

Bir iş kaleminde belirsizlik ortaya çıktığında sorumluluk iki taraftır, ancak nitelikleri farklıdır:

- **İşi tanımlayan taraf:** Kabul ölçütlerini, kapsamı ve bilinen kısıtları açıkça belirtmekle yükümlüdür.
- **İşi üstlenen taraf:** Fark ettiği belirsizlikleri görünür kılmak, varsayımlarını açıkça kaydetmek ve gerektiğinde durdurmakla yükümlüdür.

Olgun bir ekipte bu iki yükümlülük birbirini tamamlar. Yükümlülüklerden birinin sistematik olarak yerine getirilmemesi, diğerinin sürekli telafi yükü altında kalmasına ve zamanla bu telafinin görünmez emek hâline gelmesine yol açar.

Sorunun Teknik Değil Kültürel Niteliği

“Bu daha açık yazılsaydı ekip daha hızlı ilerlerdi” biçimindeki bir ifade, niyet bakımından bir süreç gözlemidir. Karşı tarafça “kişisel eleştiri” olarak yorumlandığında sorun teknik alandan çıkarak geri bildirim kültürüne taşınır.

Detert ve Edmondson’ın (2011) çalışması, çalışanların sessiz kalma davranışının çoğunlukla somut bir olumsuz deneyimden değil, örtük ses kuramlarından (implicit voice theories) kaynaklandığını göstermektedir: üstün önünde olumsuz konuşmanın uygunsuz olduğu, verinin yeterince güçlü olmadıkça konuşulmaması gerektiği yönündeki içselleştirilmiş inançlar, açıkça bir yaptırım bulunmasa dahi sessizlik üretmektedir.

Olgun ekipte beklenen tepki, savunma değil süreç sorusudur: “Bunu bir sonraki seferde nasıl daha iyi yapabiliriz?” Bu tepkiyi mümkün kılan koşul, Edmondson’ın (1999) tanımladığı *psikolojik güvenlik*ti: ekip üyelerinin, kişilerarası risk taşıyan davranışların (soru sormak, hata kabul etmek, itiraz etmek) cezalandırılmayacağına dair paylaşılan inancı. Aynı bulgu, Google’ın ekip etkinliği üzerine yürüttüğü Project Aristotle çalışmasında da en güçlü açıklayıcı değişken olarak doğrulanmıştır (Rozovsky, 2015).

Tekil ve Sistematik Belirsizlik Ayırımı

Temel ayırım şudur:

- **İyi mühendis** belirsizliği yönetir.
- **İyi ekip** aynı belirsizliği sürekli yeniden üretmez.

Bu ayırımın kuramsal karşılığı Shewhart’ın (1931) ve Deming’in (1986) kalite yönetiminde kurduğu *özel nedenli* ile *ortak nedenli* değişkenlik ayırımıdır. Özel nedenli değişkenlik tekil ve atfedilebilirdir; yerinde müdahaleyle çözülür. Ortak nedenli değişkenlik ise sistemin kendi davranışındır; bireye atfedilemez ve yalnızca süreç değiştirilerek azaltılabilir.

Deming’in vurgusu burada belirleyicidir: ortak nedenli değişkenliğe bireysel müdahaleyle karşılık verilmesi, sistemi iyileştirmediği gibi değişkenliği artırır. Tekrarlayan belirsizliğin bireysel gayretle karşılanması bu nedenle yalnızca yorucu değil, yöntemsel olarak da hatahdır. Bir belirsizlik örüntüsünün üçüncü kez yinelenmesi, artık bireysel yönetim değil süreç tasarımı konusudur.

Tek Döngülü ve Çift Döngülü Öğrenme

Argyris’in (1977) ayırımı bu tabloyu tamamlar. *Tek döngülü öğrenme*, mevcut hedefler ve varsayımlar sabit tutularak hatanın düzeltilmesidir; belirsizliğin o örnek için giderilmesi bu kategoriye girer. *Çift döngülü öğrenme*, hatanın kendisini üreten hedeflerin, ölçütlerin ve varsayımların sorgulanmasıdır; belirsizliği üreten süreç tasarımının değiştirilmesi bu kategoriye girer.

Argyris’in en önemli bulgusu, örgütlerin çift döngülü öğrenmeye karşı savunmacı rutinler geliştirdiğidir. Bu rutinler mahcubiyet ve tehdidi önlemek üzere

işler; ancak bunu yaparken kendilerinin tartışılmasını da engeller. Sonuç, sorunun konuşulamaz olması ve konuşulamaz oluşunun da konuşulamaz olmasıdır.

Toyota üretim sisteminin *jidoka* ve andon uygulamaları, çift döngülü öğrenmenin kurumsallaşmış bir örneğidir: hattı durdurma yetkisinin en alt düzeye verilmesi, tekil sorunun kök nedene kadar izlenmesini zorunlu kılar (Ohno, 1988; Rother, 2009).

Batık Maliyet ve Taahhüt Tırmanması

Bir çalışmanın sürdürülmesine ilişkin rasyonel ölçüt yalnızca beklenen gelecek getirisi olmasına karşın, karar vericiler geçmiş harcamayı da hesaba katma eğilimindedir. Arkes ve Blumer (1985) bu eğilimi deneysel olarak göstermiş; Staw (1976) ise başarısız bir eylem dizisine ek kaynak ayırma davranışını *taahhüt tırmanması* olarak kavramsallaştırmıştır.

Tırmanmayı güçlendiren etkenler arasında kararın kişisel sorumluluğunun yüksek olması, kararın kamuya açık biçimde savunulmuş olması ve geri dönüşün beceriksizlik olarak yorumlanacağı beklentisi bulunur. Bu üç etkenin tamamı örgütsel olarak azaltılabilir niteliktedir; dolayısıyla batık maliyet tuzağı yalnızca bireysel bir zayıflık değil, tasarlanabilir bir örgütsel koşuldur.

Çıkmaz Sokağın Erken Tanısı

Bir özellik geliştirilirken her adımda yeni istisna ortaya çıkıyor, çözüm giderek daha karmaşık ve kırılgan hâle geliyor ve her düzeltme yeni bir özel durum gerektiriyorsa, bu bir tam işaretidir. Bölüm II’de anılan Lakatos ölçütü doğrudan uygulanabilir: yapılan tadiller yeni öngörü üretmiyor, yalnızca mevcut sorunu kapatıyorsa, izlenen yol gerileyen bir programdır.

Güçlü mühendislik refleksi bu noktada, o ana kadar harcanan emeğe rağmen yönü değiştirme kararı üretir. Deneyimsiz refleks ise harcanan emeği gerekçe göstererek devam eder ve kaybı büyütür. Mesele doğru yolu seçmek kadar, yanlış yolda olunduğunu erken kabul edebilmektir.

Weick’in (1996) Mann Gulch yangını üzerine yaptığı çözümleme bu davranışın uç bir örneğini sunar: itfaiyecilerin kaçış sırasında ağır ekipmanlarını bırakmamaları, yalnızca fiziksel bir hata değil kimlik meselesidir; araçlar bırakıldığında geriye kalan “kim olunduğu” sorusu belirsizleşir. Weick’in çıkardığı ders, araçları bırakabilmenin öğrenilebilir ve öğretilebilir bir yetkinlik olduğudur.

Bu yetkinliğin süreç düzeyindeki karşılıkları:

- Çalışma öncesinde başarısızlık senaryosunun yazılması (Klein’in (2007) önerdiği *ön-mortem* yöntemi)
- Terk ölçütünün önceden ve nicel olarak belirlenmesi
- Zaman kutulu araştırma (timeboxed spike) uygulaması
- Kararın kişiye değil kanıta bağlanacak biçimde belgelenmesi
- Yön değiştirmenin başarısızlık değil bulgu olarak raporlanması

Klein'in (1998) uzman karar verme üzerine çalışmaları, deneyimli kişilerin seçenekleri karşılaştırarak değil, örüntü tanıyarak karar verdiğini göstermektedir. Çıkmaz sokak sezgisi de bu türden bir örüntü tanımadır; dolayısıyla aktarılabirliği vaka anlatımı ve birlikte çalışma yoluyla sağlanır, kural listeleriyle değil.

Suçlamasız İnceleme ve Adil Kültür

Yön değiştirme kararının maliyetsiz hâle gelmesi, hata sonrası inceleme kültürüne bağlıdır. Dekker'in (2007) *adil kültür* çerçevesi, kim sorumludur sorusundan hangi koşullar bu davranışı makul kıldı sorusuna geçişi önerir. Allspaw'ın (2012) suçlamasız olay incelemesi uygulaması, aynı ilkeyi yazılım işletimi bağlamında kurumsallaştırmıştır: amaç, olay anında bilgi sahibi olan kişinin o an neyi neden makul gördüğünü öğrenmektir.

Bu çerçevenin tersine mühendislik bölümüyle bağlantısı doğrudandır. İkisi de aynı soruyu sorar: gözlemlenen davranışı üreten iç durum neydi? Fark yalnızca incelenen sistemin bir yazılım mı yoksa bir ekip mi olduğudur.

Ego, Kapsam ve Etki Alanı

İyi mühendislik uygulaması, kişinin kendi modeline değil kanıta bağlanmasını gerektirir. Bu, kişisel değersizleştirme anlamına gelmez; bağlanma nesnesinin kimlikten işe kaydırılmasıdır.

İkinci olarak, bu bölümde ele alınan meselelerin tamamı kolektiftir. Conway'nin (1968) yasası, bir sistemin yapısının onu üreten örgütün iletişim yapısını yansıttığını ortaya koyar; buradan çıkan sonuç, bazı teknik sorunların bireysel çabayla değil ancak iletişim yapısı değiştirilerek çözülebileceğidir. Dolayısıyla etki alanının bilinçli seçimi, kaçınma değil bir mühendislik kararıdır: değiştirilebilir olan ile yalnızca katlanılabilir olanın ayrımı, kaynak tahsisi problemidir.

Yetkinliğin Tanımı

Bu bölümde ele alınan yetkinlik üç bileşenden oluşmaktadır. Birincisi *belirsizlikle çalışabilme*: eksik tanımlı bir görevde ilerlemeyi tümüyle durdurmadan, bilinen ile varsayılanı ayırt ederek çalışabilme. İkincisi *belirsizliğe açıklık kazandırma*: örtük kalmış kabulleri, açıkça ifade edilmiş soru veya varsayım hâline getirme. Üçüncüsü *baskı altında karar alabilme*: hızlanma gerektiren anda, bilginin eksik kalacağını kabul ederek yeterince doğru kararı zamanında üretebilme.

Üçünün ortak paydası, belirsizliğin ortadan kaldırılması gereken bir kusur değil, yönetilmesi gereken bir çalışma koşulu olarak ele alınmasıdır. Simon'un (1956) sınırlı akılcılık çerçevesi bu tutumun zeminini oluşturur: karar verici, tam bilgi varsayımı altında değil, bilgi ve zaman kısıtı altında yeterli çözümü arar.

Kurumsal Kas Kavramı

Bir ekibin belirsizlikle başa çıkma kapasitesi, bireylerin yetkinliklerinin toplamına indirgenemez. Yinelenen bir güçlüğün her seferinde yeniden bireysel çabayla aşılması ile aynı güçlüğün süreç düzeyinde ortadan kaldırılması, farklı kapasite türleridir. İkincisi burada *kurumsal kas* olarak adlandırılmaktadır: ekibin, daha önce karşılaştığı bir belirsizlik biçimini tekrar üretmeyecek şekilde kendi çalışma biçimini değiştirebilme yeteneği.

Kavramın ayırt edici özelliği, kişilerden bağımsız olarak varlığını sürdürmesidir. Bireysel yetkinlik ekipten ayrılmayla birlikte kaybolur; kurumsal kas ise şablonlara, kabul ölçütlerine, kontrol listelerine ve paylaşılan normlara yerleştiği ölçüde kalıcıdır. Nelson ve Winter'ın (1982) örgütsel rutin kavramı bu kalıcılığın taşıyıcısını tanımlar: rutin, örgütün belleğinin fiilî biçimidir.

Bireysel Telif ile Norm Aktarımı Arasındaki Seçim

Eksik tanımlı bir iş kalemiyle karşılaşan taraf için iki farklı eylem yolu bulunmaktadır:

- **Bireysel telif:** Belirsizliğin sessizce giderilmesi; eksik tanımın kişisel çaba, tahmin ve enformel sorularla kapatılması.
- **Norm aktarımı:** Geri bildirimin, kişiye yönelik bir eleştiri olarak değil, iş kaleminin nasıl yazılması gerektiğine ilişkin paylaşılan bir ölçüt önerisi olarak iletilmesi.

Birinci yol kısa vadede daha hızlıdır ve çatışma üretmez; ancak eksikliği görünmez kıldığı için kaynağı korur. Bu, Bölüm VII'de ele alınan tek döngülü öğrenmenin tipik biçimidir ve zamanla telif eden tarafta biriken görünmez emeğe dönüşür. İkinci yol kısa vadede maliyetlidir, çünkü savunmacı tepki riski taşır; buna karşılık kalıcı etkisi olan tek yoldur.

Seçimin ölçütü tekrar sayısıdır. Tekil bir eksiklik bireysel telifle kapatılabilir; aynı eksikliğin yinelenmesi hâlinde bireysel telif, sistem sorununu maskeleyen bir mekanizmaya dönüşür.

Bölüm VII: Analiz Direncine Karşı Tersine Mühendislik

Analiz Direncinin Tasarım Nesnesi Hâline Gelmesi

Önceki bölümlerde sistem, gözlemciye karşı yansız bir nesne olarak ele alınmıştır. Ticari ve güvenlik açısından hassas ürünlerin önemli bir bölümünde ise durum farklıdır: analiz maliyetinin artırılması, açık bir tasarım hedefidir. Bu durumda tersine mühendislik, sabit bir sistemin çözümlenmesi değil, bir tasarımcıyla girilen asimetrik bir etkileşim hâlini alır.

Anderson ve Kuhn'un (1996) kurcalama direnci üzerine çalışması, bu alandaki temel bulguyu ortaya koymuştur: mutlak koruma bulunmamaktadır; koruma düzeyleri, saldırganın erişebildiği ekipman ve harcayabileceği süre cinsinden tanımlanabilir. Buradan çıkan sonuç, korumanın imkânsızlık değil maliyet

ürettiğidir. Aynı yazarların düşük maliyetli saldırılar üzerine çalışması (1997), yüksek maliyetli laboratuvar donanımı gerektirdiği varsayılan pek çok saldırının çok daha düşük bütçelerle yinelenemediğini göstermiştir.

Kerckhoffs ilkesinin uzantısı burada geçerlidir: tasarımın gizliliğine dayanan koruma, gizlilik ihlal edildiğinde tümüyle çöker. Dolayısıyla karartma ve fiziksel koruma, güvenliğin kendisi değil, saldırıyanın zaman bütçesini tüketen bir katman olarak konumlandırılmalıdır.

Yaygın Koruma Mekanizmaları

Analiz direncinin başlıca biçimleri şunlardır:

- **Anahtar silme (zeroization):** Fiziksel müdahale algılandığında kriptografik anahtarların geri döndürülemez biçimde silinmesi. FIPS 140-3 standardı, fiziksel güvenlik düzeylerini ve silme gereksinimlerini derecelendirilmiş biçimde tanımlamaktadır.
- **Güvenli açılış:** Yürütülecek her katmanın bir öncekinde imzasının doğrulanması yoluyla güven zinciri kurulması (Arbaugh, Farber ve Smith, 1997).
- **Cihaza bağlı sırlar:** Yazılımın veya verinin, yalnızca belirli bir donanım örneğine ait benzersiz anahtarla çözülebilmesi; böylece belleğin fiziksel olarak kopyalanması tek başına yetersiz kalır.
- **Kurcalama kanıtı ve kurcalama direnci:** Birincisi müdahalenin sonradan fark edilmesini, ikincisi müdahalenin fiziksel olarak zorlaştırılmasını hedefler. İkisi farklı tehdit modellerine karşılık gelir.
- **Sırların dağıtılması:** Kritik bilginin tek bir bileşende toplanmaması; bütünün yeniden kurulması için birden fazla alt sisteme eş zamanlı erişim gerekmesi.

Bütün bir sistemin müdahale algısıyla kendini işlevsiz kılması, tasarım açısından genellikle tercih edilmez. Yanlış pozitif oranı sıfırlanamayacağı için, bakım işlemi veya arızalı bir sensör nedeniyle tetiklenecek geri alınamaz bir yıkım, korumadan daha büyük bir kullanılabilirlik riski üretir. Uygulamada koruma, bütünün imhası biçiminde değil, kritik veri ve anahtar düzeyinde kademeli olarak tasarlanır.

Yarı İletken Düzeyinde Üç Eşik

Bir tümleşik devrenin fiziksel olarak ele geçirilmiş olması, işlevinin anlaşıldığı anlamına gelmez; işlevinin anlaşılmış olması da üretilebileceği anlamına gelmez. Torrance ve James'in (2009) derlemesi ile Quadir ve diğerlerinin (2016) taraması, bu ayrımı üç ayrı yetkinlik eşiği olarak ortaya koymaktadır:

$$\text{İnceleyebilmek} \neq \text{Tasarlayabilmek} \neq \text{Üretebilmek}$$

Birinci eşik, kapsül açımı, katman katman aşındırma, görüntüleme ve netlist çıkarımı yoluyla devrenin topolojisinin elde edilmesidir. Skorobogotov'un (2005) yarı-girişimsel saldırılar üzerine çalışması, bu aşamada kullanılan tekniklerin sınıflandırmasını sunar.

İkinci eşik, elde edilen topolojinin işlevsel niyete çevrilmesidir. Milyarlarca anahtarlama elemanı içeren bir tasarımda, çıkarılan bağlantı listesinin anlamlı modüllere ayrıştırılması başlı başına bir araştırma problemidir; sonuç, tasarımcının kullandığı üst düzey tanımın kendisi değil, onun kayıplı bir izdüşümüdür.

Üçüncü eşik, üretim yetkinliğidir ve ilk ikisinden bağımsızdır. Tasarımın tümüyle bilinmesi, onu üretecek süreç kontrolüne, verimlilik eğrisine ve tesis altyapısına sahip olmayı sağlamaz.

Bileşim ile Üretilabilirlik Arasındaki Boşluk

Aynı yapı malzeme düzeyinde de görülür. Bir numunenin bileşimi analitik yöntemlerle belirlenebilir; buna karşılık aşağıdaki bilgiler numuneden doğrudan okunamaz:

- Ham maddenin saflaştırma yordamı
- Uygulanan sıcaklık ve basınç dizisi ile bunların zamanlaması
- Üretimde tutulan tolerans aralıkları ve süreç kontrol sınırları
- Yüzey işlemlerinin türü ve uygulanma sırası
- Kabul edilen ve reddedilen parti oranları
- Uzun süreli kullanımda gözlemlenen yaşlanma ve yorulma davranışı

Bohn'un (1994) teknolojik bilgi düzeyleri ölçeği bu boşluğu işlemselleştirir: bir değişkenin varlığını bilmek, onu ölçebilmek, etkisini niceliksel olarak modelleyebilmek ve süreci o model üzerinden kontrol edebilmek birbirinden ayrı düzeylerdir. Ele geçirilen bir örnek, çoğunlukla yalnızca ilk düzeyi sağlar.

Bu boşluğun kuramsal adı örtük bilgidir (Polanyi, 1966). Cowan, David ve Foray (2000), bilginin kodlanabilir kısmı ile kodlanamayan kısmı arasındaki sınırın sabit olmadığını, ancak her teknolojiye belirli bir kodlanmamış artığın kaldığını göstermiştir. Von Hippel'in (1994) yapışkan bilgi kavramı, bu artığın aktarım maliyetinin neden yüksek olduğunu açıklar.

Ürün ile Tamamlayıcı Varlıklar

Karmaşık bir ürünün işlevselliği, çoğu durumda ürünün kendisinde değil, onu çevreleyen varlık kümesinde bulunur: yapılandırma ve planlama araçları, bakım ve teşhis yazılımları, kalibrasyon donanımı, yedek parça ve envanter sistemi, eğitim materyali ve benzetim ortamı, güncelleme altyapısı, tedarikçi ağı ve yıllar içinde biriken arıza verisi.

Teece'nin (1986) tamamlayıcı varlıklar çerçevesi, bir yeniliğin değerinin büyük ölçüde bu varlıklara erişimle belirlendiğini ortaya koyar. Pisano ve Shih'in (2009) sınavi müşterek (industrial commons) kavramı ise aynı olguyu üretim tarafında tanımlar: bir ürünün yeniden üretilebilirliği, tek bir firmanın değil, bölgesel tedarikçi, yetkin işgücü ve süreç bilgisi yığınının varlığına bağlıdır.

Henderson ve Clark'ın (1990) bileşen bilgisi ile mimari bilgi ayrımı bu tabloya doğrudan uygulanabilir. Tersine mühendislikle elde edilen bilginin büyük bölümü bileşen düzeyindedir; bileşenlerin hangi gerekçeyle o biçimde birleştirildiğine ilişkin mimari bilgi ise büyük ölçüde örtük kalır ve yalnızca ürünün tasarım tarihçesiyle birlikte anlaşılabilir.

Bu bölümün sonucu şu biçimde özetlenebilir:

Ürün+Üretimsüreci+Malzemebilgisi+Testverisi+Tedarikzinciri+Örtükustalık

Ele geçirilen örnek yalnızca ilk terimdir; kalan terimler bağımsız olarak inşa edilmedikçe yeniden üretim kabiliyeti oluşmaz.

Destek Bağımlılığı ve Kademeli Kabiliyet Kaybı

Bir sistemin işlevselliği yalnızca ilk teslim anında değil, sürekli beslenen girdilerle sürdürülür: yazılım güncellemeleri, yapılandırma ve veri dosyaları, kalibrasyon değerleri, yedek parça akışı ve sertifika yenilemeleri. Bu girdilerin kesilmesi, sistemi ani biçimde durdurmaz; kabiliyeti kademeli olarak eritir.

Destekkesilmesi → Verieskimesi → Bakımgüçlüğü → Parçakıtlığı → Kabiliyetkayı

Bu yapı, tersine mühendisliğin hangi durumlarda zorunlu hâle geldiğini de açıklar. Tedarikçinin desteği sonlandırdığı, kaynak kodun erişilemez olduğu veya sözleşmenin sona erdiği durumlarda, sistemin sürdürülebilmesi için davranışının bağımsız olarak modellenmesi tek seçenek hâline gelir. Bu, Bölüm IV'te tanımlanan hedef kümesinin en yaygın gerçek hayat örneğidir.

Bir sistemin sahibinin dahi bağımsız sürdürülebilirlik için gerekli teknik veriye sahip olmayabileceği gözlemi, ayrıca satın alma aşamasında teknik veri haklarının müzakere edilmesinin neden kritik olduğunu göstermektedir.

Bölüm VIII: Tersine Mühendisin Ayırt Edici Bilgisi

Yöntem Bilgisi ile Alan Bilgisi Ayrımı

Bu alanda yetkin kişileri ayıran şey, ortalamadan daha geniş bir alan bilgisine sahip olmaları değildir. Ayırt edici olan, bilinmeyen bir sistem karşısında hangi kanıtın toplanacağını, hangi deneyin kurulacağını ve üretilen çıkarımın hangi güven düzeyini hak ettiğini bilmeleridir. Yani sahip olunan şey bir ürün bilgisi değil, bilgi üretme yöntemidir.

Bu bölümde, önceki bölümlerdeki yöntemleri tamamlayan ve daha çok bilişsel alışkanlık niteliği taşıyan ayırt edici unsurlar ele alınmaktadır.

Davranıştan Zorunlu Yapıya Çıkarım

Kullanıcı, ürünün ne yaptığını gözlemler. Analist ise şunu sorar: gözlenen işlevin gerçekleşebilmesi için içeride hangi bileşenlerin bulunması *zorunludur*?

Gözlenendavranış → Zorunlu yapısalönkosullar

Bu çıkarım biçimi, olası açıklamaların tümünü değil, hiçbir açıklamada eksik olamayacak asgari yapıyı hedefler. Örneğin bir bileşenin belirli bir gecikme sınırının altında yanıt üretmesi, işlemin uzak bir kaynaktan yapılmadığını; belirli bir yeniden başlatma sonrasında durumun korunması, kalıcı bir saklama katmanının bulunduğunu zorunlu kılar. Bu tür çıkarımlar, olasılık değil kısıt üretir ve bu nedenle hipotez uzayını diğer gözlemlerden daha hızlı daraltır.

Dennett'in (1987) tasarım tutumu kavramı bu tarzın felsefi karşılığıdır: yapay bir nesne, mekanik ayrıntısına inilmeden, tasarlanmış olduğu varsayımı altında ve amacına göre öngörülebilir.

Sistemlerin İz Bırakması

Çalışan her sistem, işlevsel arayüzünün dışında da gözlemlenebilir izler üretir: zamanlama, güç tüketimi, ısı davranış, akustik yayılım, elektromanyetik yayılım, bellek kullanım profili, dosya erişim örüntüsü, ağ trafiği hacmi ve mekanik aşınma.

Bu izlerin kanıt değeri taşıdığı gözlemi, yan kanal analizi yazınının temelini oluşturur (Kocher, 1996; Kocher, Jaffe ve Jun, 1999). Genkin, Shamir ve Tromer'in (2014) akustik yayılımdan anahtar çıkarımı üzerine çalışması, kanalın ne kadar dolaylı olabileceğini göstermektedir.

Pratikteki ayrım şu biçimdedir: sıradan gözlemci "çalışmadı" kaydını tutar; analist ise hangi aşamada, ne kadar süre sonra, hangi kaynak tüketimiyle ve sistemin hangi gözlemlenebilir durumu değişerek çalışmadığını kaydeder. Birinci kayıt tek bitlik bilgi taşır, ikincisi hipotez ayırt eder.

Arayüz Önceliği

Bir sistemin bileşenlerini anlamının en verimli yolu, çoğu zaman bileşenlerin kendisi değil, aralarındaki iletişimidir. Arayüze yöneltilen sorular şunlardır: bu bileşene ne girmektedir, ne çıkmaktadır, veri hangi biçimdedir, hangi olay hangi bileşeni tetiklemektedir, bileşen devre dışı bırakıldığında hangi işlevler kaybolmaktadır?

Bu önceliğin kuramsal gerekçesi Parnas'ın (1972) bilgi gizleme ilkesidir: iyi tasarlanmış bir bileşenin iç yapısı bilinçli olarak gizlenmiş, buna karşılık arayüzü açıkça tanımlanmıştır. Dolayısıyla tasarımın en çok bilgi taşıyan yüzeyi, gizlenmemiş olan yüzeydir. Baldwin ve Clark (2000) aynı olguyu modüler tasarım kuralları çerçevesinde formelleştirir: mimarinin gerçek yapısı, arayüz tanımlarında kodlanmıştır.

Kısıtlardan Tasarım Niyetine

Her tasarım, birbiriyle çatışan kısıtların uzlaştırılmasının sonucudur: maliyet, ağırlık, enerji, ısı, hız, güvenlik, bakım kolaylığı, üretilebilirlik, dayanıklılık ve sertifikasyon gereksinimleri.

Bu nedenle biçimden niyete giden en verimli soru, “bu neden böyle yapılmış?” değil, “tasarımcı bunu hangi kısıt nedeniyle böyle yapmak *zorunda* kalmış?” biçimindedir. Alexander’ın (1964) biçim–bağlam uyumu çözümlemesi ve Vincenti’nin (1990) mühendislik bilgisinin doğası üzerine incelemesi, tasarımın bir tercihler dizisi değil, kısıt karşılama kayıtları bütünü olarak okunabileceğini göstermektedir.

Bu okumanın pratik sonucu, alışılmadık görünen bir çözümün genellikle beceriksizlik değil, görünmeyen bir kısıtın izi olmasıdır. Kısıtın belirlenmesi, tasarım hakkında bileşenin kendisinden daha fazla bilgi verir.

“Ne Yapılmış” ile “Nasıl Üretilmiş” Ayrımı

Bir bileşenden doğrudan okunabilenler ile okunamayanlar sistematik olarak ayrılmalıdır:

Okunabilen	Okunamayan
Biçim ve boyut	Üretim sırası
Malzeme ailesi	İşlem sıcaklıkları ve süreleri
İşlev	Kalite kontrol ölçütleri
Yaklaşık tolerans	Başarısız deneme sayısı
Bağlantı biçimi	Kullanılan özel tezgâhlar
Yüzey durumu	Operatör düzeyindeki ustalık

Mansfield, Schwartz ve Wagner’in (1981) taklit maliyetleri üzerine ampirik çalışması bu ayrımı niceliksel olarak destekler: taklit maliyeti, özgün geliştirme maliyetinin önemli bir oranında kalmaktadır; yani ürünün varlığı, onu yeniden üretme maliyetini sifıra yaklaştırmaz. Levin ve diğerlerinin (1987) sahip-enebilirlik araştırması, sektörlerin önemli bölümünde üretim sürecindeki üstünlüğün patentten daha etkili bir koruma sağladığını göstermektedir; bu bulgu, korunan asıl varlığın çoğu zaman tasarım değil süreç olduğunu ima eder.

Arızanın Bilgi Kaynağı Olarak Kullanılması

Sorunsuz çalışan bir sistem iç yapısını gizler; arıza ise onu açığa çıkarır. Belirli bir bileşenin devre dışı kalmasıyla hangi işlevlerin kaybolduğu, hangilerinin sürdüğü ve hangi bileşenlerin birbirine bağımlı olduğu, olağan çalışma sırasında elde edilemeyecek bir bağımlılık haritası üretir.

Petroski’nin (1985) mühendislik tarihinden çıkardığı temel gözlem budur: başarısızlık, başarıdan daha fazla yapısal bilgi taşır. Aynı ilkenin denetimli biçimde uygulanması iki ayrı yönetsel geleneğe kaynaklık etmiştir. Birincisi, hesaplama kasıtlı hata enjekte ederek iç durumu açığa çıkaran hata saldırıları çerçevesidir (Boneh, DeMillo ve Lipton, 1997; Bar-El ve diğerleri,

2006). İkincisi, üretim sistemine denetimli bozulma uygulayarak gerçek bağımlılıkları ortaya çıkaran kaos mühendisliği uygulamasıdır (Basiri ve diğerleri, 2016).

Ortak ilke şudur: arıza, sistemin en yüksek bilgi verimine sahip çalışma noktasıdır ve bu nedenle kaçınılacak değil, denetim altında üretilecek bir durumdur.

Disiplinlerarası Bağlama Yeteneği

Karmaşık bir yapının çözümlenmesi genellikle tek bir uzmanlık alanının sınırlarını aşar: bir elektronik kartın davranışı devre, gömülü yazılım, sinyal işleme, ısı davranış, üretim tekniği ve malzeme bilgisinin kesişiminde bulunur.

Buradaki ayırt edici yetkinlik, tüm alanların derinlemesine bilinmesi değildir; hangi gözlemin hangi disiplinin sorusu olduğunun ayırt edilebilmesidir. Bu, sınır nesneleri (boundary objects) yazınında tanımlanan role karşılık gelir: farklı uzmanlık toplulukları arasında ortak bir temsil kurabilen kişi, kendisi uzman olmasa da bilgi akışını mümkün kılar (Star ve Griesemer, 1989). Bu yetkinliğin ölçütü, üretilen soruların ilgili uzman tarafından yanıtlanabilir biçimde formüle edilebilmesidir.

“Çalışıyor” ile “Anladım” Ayrımı

Bir davranışın taklit edilmesi ile üretilme gerekçesinin anlaşılması farklı başarılarıdır. Gözlem kümesindeki tüm girdilerde doğru çıktı üreten bir model, kümenin dışına çıkıldığında bozuluyorsa, ilgili örüntü anlaşılmamış yalnızca ezberlenmiştir. Bu, Bölüm IV’te tanımlanan bağımsız doğrulama ölçütünün bilişsel karşılığıdır.

Bölümün özeti şu biçimde formüle edilebilir:

$$\text{Ayırt edici bilgi} = \text{cevabı bilmek değil, cevabı sistemden çıkarmayı bilmek}$$

Bu yetkinliğin gözlemlenebilir bileşenleri merak, sabır, deney tasarımı, ölçüm disiplini, denetimli şüphecilik ve sistem düşüncesidir. Hiçbiri tek başına yeterli değildir; ayırt edici olan, altısının aynı problem üzerinde sırayla uygulanabilmesidir.

Bölüm IX: Sentez

İki bölümde incelenen alanlar, aynı epistemik disiplinin farklı nesnelere yöneltilmiş hâlleridir. Yapısal karşılık şu biçimde özetlenebilir:

İlke	Sisteme uygulanışı	Örgüte uygulanışı
Gözlem–yorum ayrımı	Bulguyu hipotezden ayırmak	Olguyu kişisel yargıdan ayırmak
Yanlışlama	“Bu açıklama nerede çöker?”	“Bu yaklaşım yanlış olabilir mi?”
Çoklu hipotez	Rakip açıklamaları taşımak	Alternatif süreç tasarımlarını taşımak
Erken terk	Gerileyen hipotezi bırakmak	Çıkamaz sokaktan geri dönmek
Tekrarın anlamı	Tekrarlayan davranış deterministiktir	Tekrarlayan belirsizlik sistemiktir
Katman disiplini	Doğru soyutlamada kalmak	Doğru müdahale düzeyini seçmek
Bağlanma nesnesi	Modele değil kanıta bağlanmak	Kimliğe değil işe bağlanmak
Belgeleme	Bilinen ile varsayılanı ayırmak	Karar ile gerekçeyi ayırmak

Tersine mühendislikte sorulan “bu açıklama hangi koşulda çöker?” sorusu ile ekip içinde sorulan “bu yol yanlışsa bunu ne zaman anlarım?” sorusu aynı yöntemsel tutumun iki uygulamasıdır. Birincisinde yanlışlama dış sisteme, ikincisinde kişinin kendi çalışmasına ve ekibin kendi süreçlerine yöneltilmektedir. İkinci uygulamanın daha zor olmasının nedeni teknik değil psikolojiktir: dış sistemin modeli çürütüldüğünde kaybedilen yalnızca bir hipotez, kişinin kendi yaklaşımı çürütüldüğünde ise kimliğinin bir parçası olarak algılanan bir yatırımdır. Bu farkı yönetilebilir kılan koşul, psikolojik güvenliğin ve suçlamasız inceleme pratiğinin kurumsal olarak tesis edilmesidir.

Nihai ölçüt her iki düzlemde de aynıdır: anlaşıldığı düşünülen şey yeniden üretilmiyorsa, henüz anlaşılmamıştır.

References

- [1] Alexander, C. (1964). *Notes on the Synthesis of Form*. Cambridge, MA: Harvard University Press.
- [2] Anderson, R. ve Kuhn, M. (1996). Tamper Resistance — A Cautionary Note. *Proceedings of the 2nd USENIX Workshop on Electronic Commerce*, 1–11.
- [3] Anderson, R. ve Kuhn, M. (1997). Low Cost Attacks on Tamper Resistant Devices. *Security Protocols Workshop*, LNCS 1361, 125–136.
- [4] Anderson, R. (2020). *Security Engineering: A Guide to Building Dependable Distributed Systems* (3. baskı). Indianapolis: Wiley.
- [5] Arbaugh, W. A., Farber, D. J. ve Smith, J. M. (1997). A Secure and Reliable Bootstrap Architecture. *Proceedings of the IEEE Symposium on Security and Privacy*, 65–71.
- [6] Baldwin, C. Y. ve Clark, K. B. (2000). *Design Rules, Volume 1: The Power of Modularity*. Cambridge, MA: MIT Press.
- [7] Bar-El, H., Choukri, H., Naccache, D., Tunstall, M. ve Whelan, C. (2006). The Sorcerer’s Apprentice Guide to Fault Attacks. *Proceedings of the IEEE*, 94(2), 370–382.

- [8] Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J. ve Rosenthal, C. (2016). Chaos Engineering. *IEEE Software*, 33(3), 35–41.
- [9] Bohn, R. E. (1994). Measuring and Managing Technological Knowledge. *Sloan Management Review*, 36(1), 61–73.
- [10] Boneh, D., DeMillo, R. A. ve Lipton, R. J. (1997). On the Importance of Checking Cryptographic Protocols for Faults. *Advances in Cryptology — EUROCRYPT '97*, 37–51.
- [11] Cowan, R., David, P. A. ve Foray, D. (2000). The Explicit Economics of Knowledge Codification and Tacitness. *Industrial and Corporate Change*, 9(2), 211–253.
- [12] Dennett, D. C. (1987). *The Intentional Stance*. Cambridge, MA: MIT Press.
- [13] Genkin, D., Shamir, A. ve Tromer, E. (2014). RSA Key Extraction via Low-Bandwidth Acoustic Cryptanalysis. *Advances in Cryptology — CRYPTO 2014*, 444–461.
- [14] Henderson, R. M. ve Clark, K. B. (1990). Architectural Innovation: The Reconfiguration of Existing Product Technologies and the Failure of Established Firms. *Administrative Science Quarterly*, 35(1), 9–30.
- [15] Levin, R. C., Klevorick, A. K., Nelson, R. R. ve Winter, S. G. (1987). Appropriating the Returns from Industrial Research and Development. *Brookings Papers on Economic Activity*, 1987(3), 783–831.
- [16] Mansfield, E., Schwartz, M. ve Wagner, S. (1981). Imitation Costs and Patents: An Empirical Study. *The Economic Journal*, 91(364), 907–918.
- [17] Nelson, R. R. ve Winter, S. G. (1982). *An Evolutionary Theory of Economic Change*. Cambridge, MA: Harvard University Press.
- [18] Petroski, H. (1985). *To Engineer Is Human: The Role of Failure in Successful Design*. New York: St. Martin's Press.
- [19] Pisano, G. P. ve Shih, W. C. (2009). Restoring American Competitiveness. *Harvard Business Review*, 87(7–8), 114–125.
- [20] Quadir, S. E., Chen, J., Forte, D., Asadizanjani, N., Shahbazmohamadi, S., Wang, L., Chandy, J. ve Tehranipoor, M. (2016). A Survey on Chip to System Reverse Engineering. *ACM Journal on Emerging Technologies in Computing Systems*, 13(1), 1–34.
- [21] Skorobogatov, S. P. (2005). *Semi-invasive Attacks: A New Approach to Hardware Security Analysis*. Technical Report UCAM-CL-TR-630, University of Cambridge.

- [22] Star, S. L. ve Griesemer, J. R. (1989). Institutional Ecology, Translations and Boundary Objects. *Social Studies of Science*, 19(3), 387–420.
- [23] Nelson, R. R. ve Winter, S. G. (1982). *An Evolutionary Theory of Economic Change*. Cambridge, MA: Harvard University Press.
- [24] Teece, D. J. (1986). Profiting from Technological Innovation. *Research Policy*, 15(6), 285–305.
- [25] Torrance, R. ve James, D. (2009). The State-of-the-Art in IC Reverse Engineering. *Cryptographic Hardware and Embedded Systems — CHES 2009*, LNCS 5747, 363–381.
- [26] Vincenti, W. G. (1990). *What Engineers Know and How They Know It*. Baltimore: Johns Hopkins University Press.
- [27] von Hippel, E. (1994). “Sticky Information” and the Locus of Problem Solving. *Management Science*, 40(4), 429–439.
- [28] Allspaw, J. (2012). *Blameless PostMortems and a Just Culture*. Code as Craft, Etsy Engineering Blog.
- [29] Angluin, D. (1987). Learning Regular Sets from Queries and Counterexamples. *Information and Computation*, 75(2), 87–106.
- [30] Argyris, C. (1977). Double Loop Learning in Organizations. *Harvard Business Review*, 55(5), 115–125.
- [31] Argyris, C. ve Schön, D. A. (1978). *Organizational Learning: A Theory of Action Perspective*. Reading, MA: Addison-Wesley.
- [32] Arkes, H. R. ve Blumer, C. (1985). The Psychology of Sunk Cost. *Organizational Behavior and Human Decision Processes*, 35(1), 124–140.
- [33] Ashby, W. R. (1956). *An Introduction to Cybernetics*. Londra: Chapman and Hall.
- [34] Box, G. E. P. (1976). Science and Statistics. *Journal of the American Statistical Association*, 71(356), 791–799.
- [35] Caballero, J., Yin, H., Liang, Z. ve Song, D. (2007). Polyglot: Automatic Extraction of Protocol Message Format Using Dynamic Binary Analysis. *Proceedings of the 14th ACM Conference on Computer and Communications Security*, 317–329.
- [36] Chamberlin, T. C. (1890). The Method of Multiple Working Hypotheses. *Science*, 15(366), 92–96.
- [37] Chikofsky, E. J. ve Cross, J. H. (1990). Reverse Engineering and Design Recovery: A Taxonomy. *IEEE Software*, 7(1), 13–17.

- [38] Collberg, C., Thomborson, C. ve Low, D. (1997). *A Taxonomy of Obfuscating Transformations*. Technical Report 148, University of Auckland.
- [39] Conway, M. E. (1968). How Do Committees Invent? *Datamation*, 14(5), 28–31.
- [40] Cousot, P. ve Cousot, R. (1977). Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, 238–252.
- [41] Cui, W., Kannan, J. ve Wang, H. J. (2007). Discoverer: Automatic Protocol Reverse Engineering from Network Traces. *Proceedings of the 16th USENIX Security Symposium*, 199–212.
- [42] Dekker, S. (2007). *Just Culture: Balancing Safety and Accountability*. Aldershot: Ashgate.
- [43] Deming, W. E. (1986). *Out of the Crisis*. Cambridge, MA: MIT Press.
- [44] Detert, J. R. ve Edmondson, A. C. (2011). Implicit Voice Theories: Taken-for-Granted Rules of Self-Censorship at Work. *Academy of Management Journal*, 54(3), 461–488.
- [45] Dijkstra, E. W. (1972). The Humble Programmer. *Communications of the ACM*, 15(10), 859–866.
- [46] Edmondson, A. C. (1999). Psychological Safety and Learning Behavior in Work Teams. *Administrative Science Quarterly*, 44(2), 350–383.
- [47] Edmondson, A. C. (2018). *The Fearless Organization*. Hoboken, NJ: Wiley.
- [48] Eilam, E. (2005). *Reversing: Secrets of Reverse Engineering*. Indianapolis: Wiley.
- [49] Heuer, R. J. (1999). *Psychology of Intelligence Analysis*. Washington, DC: Center for the Study of Intelligence, CIA.
- [50] Josephson, J. R. ve Josephson, S. G. (1994). *Abductive Inference: Computation, Philosophy, Technology*. Cambridge: Cambridge University Press.
- [51] Kent, S. (1964). Words of Estimative Probability. *Studies in Intelligence*, 8(4), 49–65.
- [52] Klein, G. (1998). *Sources of Power: How People Make Decisions*. Cambridge, MA: MIT Press.
- [53] Klein, G. (2007). Performing a Project Premortem. *Harvard Business Review*, 85(9), 18–19.

- [54] Kocher, P. C. (1996). Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. *Advances in Cryptology — CRYPTO '96*, 104–113.
- [55] Kocher, P., Jaffe, J. ve Jun, B. (1999). Differential Power Analysis. *Advances in Cryptology — CRYPTO '99*, 388–397.
- [56] Lakatos, I. (1970). Falsification and the Methodology of Scientific Research Programmes. I. Lakatos ve A. Musgrave (Ed.), *Criticism and the Growth of Knowledge* içinde (91–196). Cambridge: Cambridge University Press.
- [57] Naur, P. (1985). Programming as Theory Building. *Microprocessing and Microprogramming*, 15(5), 253–261.
- [58] Ohno, T. (1988). *Toyota Production System: Beyond Large-Scale Production*. Cambridge, MA: Productivity Press.
- [59] Parnas, D. L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12), 1053–1058.
- [60] Parnas, D. L. ve Clements, P. C. (1986). A Rational Design Process: How and Why to Fake It. *IEEE Transactions on Software Engineering*, SE-12(2), 251–257.
- [61] Platt, J. R. (1964). Strong Inference. *Science*, 146(3642), 347–353.
- [62] Polanyi, M. (1966). *The Tacit Dimension*. Londra: Routledge.
- [63] Popper, K. R. (1959). *The Logic of Scientific Discovery*. Londra: Hutchinson.
- [64] Rother, M. (2009). *Toyota Kata: Managing People for Improvement, Adaptiveness, and Superior Results*. New York: McGraw-Hill.
- [65] Rozovsky, J. (2015). *The Five Keys to a Successful Google Team*. re:Work, Google.
- [66] Shewhart, W. A. (1931). *Economic Control of Quality of Manufactured Product*. New York: D. Van Nostrand.
- [67] Sikorski, M. ve Honig, A. (2012). *Practical Malware Analysis*. San Francisco: No Starch Press.
- [68] Simon, H. A. (1956). Rational Choice and the Structure of the Environment. *Psychological Review*, 63(2), 129–138.
- [69] Simon, H. A. (1962). The Architecture of Complexity. *Proceedings of the American Philosophical Society*, 106(6), 467–482.
- [70] Staw, B. M. (1976). Knee-Deep in the Big Muddy: A Study of Escalating Commitment to a Chosen Course of Action. *Organizational Behavior and Human Performance*, 16(1), 27–44.

- [71] Tetlock, P. E. ve Gardner, D. (2015). *Superforecasting: The Art and Science of Prediction*. New York: Crown.
- [72] Vaandrager, F. (2017). Model Learning. *Communications of the ACM*, 60(2), 86–95.
- [73] Weick, K. E. (1996). Drop Your Tools: An Allegory for Organizational Studies. *Administrative Science Quarterly*, 41(2), 301–313.
- [74] Weiser, M. (1981). Program Slicing. *Proceedings of the 5th International Conference on Software Engineering*, 439–449.
- [75] Zeller, A. ve Hildebrandt, R. (2002). Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering*, 28(2), 183–200.
- [76] Zeller, A. (2009). *Why Programs Fail: A Guide to Systematic Debugging* (2. baskı). Burlington, MA: Morgan Kaufmann.